OLTP vs OLAP

row vs column stores, warehouses, ETL vs ELT

Watch the 4-min video

In one line: OLTP is many small transactions on a few rows; OLAP is a few huge scans and aggregations. Keep them apart: run the app on a row store, and copy data (ETL or ELT) into a columnar warehouse for analytics.

1 The Monday-morning slowdown

Every Monday at nine, the finance team runs its revenue report straight on the production database. The query scans two years of orders. And every Monday at nine, checkout slows to a crawl. The report is fine. It is just running in the **wrong place**.

2 Two kinds of workload

OLTP (online transaction processing) is your app: many small, fast transactions, such as “place this order” or “get this user”. Each touches a few rows by key, in milliseconds.

OLAP (online analytical processing) is reporting: few, huge queries that scan millions or billions of rows and aggregate them, such as “total revenue by month and country”.

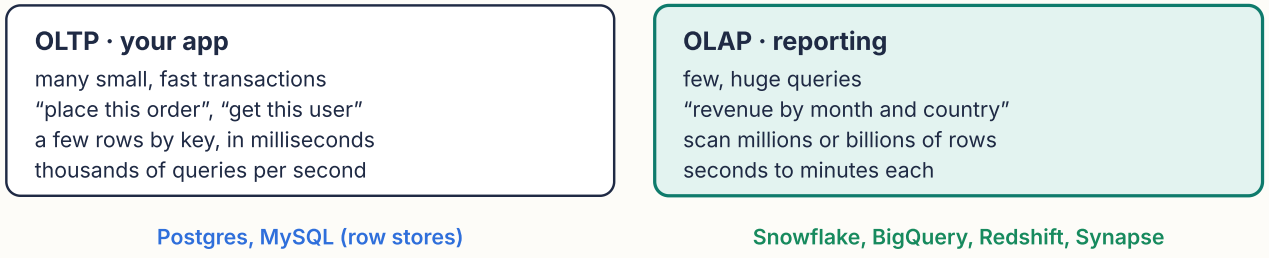


Figure 1. Same data, very different access patterns. One engine cannot be great at both.

3 Row stores vs column stores

A **row store** (Postgres, MySQL) keeps each row together on disk. That is perfect for fetching or updating one order: one read gets every field.

A **column store** keeps each column together. A report that sums one column across a billion rows reads **only that column**. Similar values sit side by side (many repeated countries, dates, small numbers), so they **compress extremely well**. Modern engines also process them in **vectorized batches**: one CPU instruction works on many values at once.

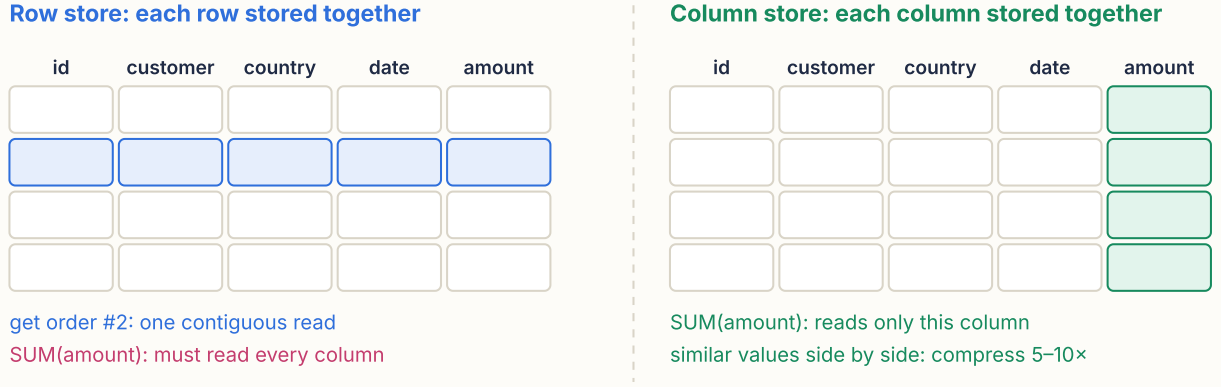


Figure 2. Highlighted cells are what each query must read. Rows suit point lookups; columns suit aggregates over a few fields.

WORKED EXAMPLE: SUMMING ONE COLUMN

An orders table has **1 billion rows × 20 columns**, about 200 bytes per row: **200 GB**. In a row store, `SUM(amount)` reads all 200 GB. In a column store it reads only `amount`: 1 billion × 8 bytes = **8 GB**, and after ~5× compression about **1.6 GB**. That is over 100× less I/O, before parallelism.

4 Data warehouses and the star schema

So analytics lives in a **data warehouse**: Snowflake, Google BigQuery, Azure Synapse or Amazon Redshift. All are columnar, and they can **scale compute separately from storage**: spin up a big cluster for the month-end report, then shrink it.

Warehouse data is often modelled as a **star schema**: a large **fact table** of events (orders, with amounts and keys) surrounded by **dimension tables** that describe them (customers, products, dates).

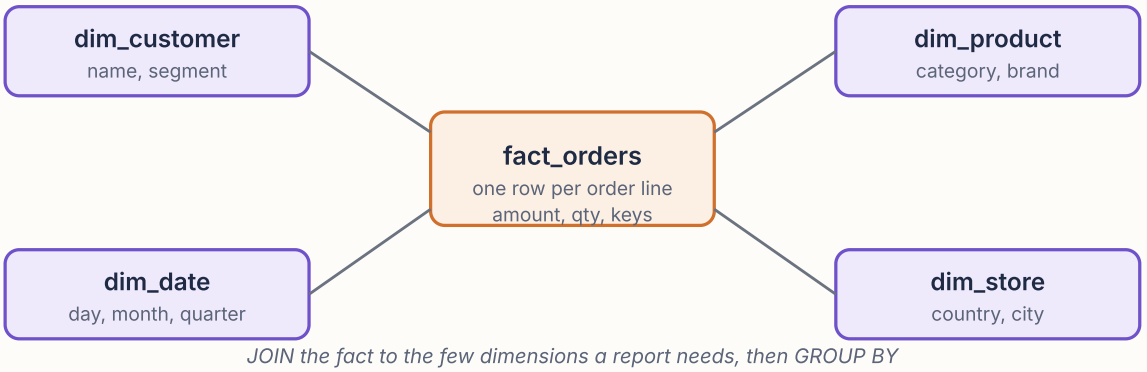


Figure 3. A star schema: one big fact table in the middle, small descriptive dimensions around it.

Why not just run analytics on the OLTP database?

- **Resource fight:** big scans compete with users for CPU, disk and memory, and push hot data out of the cache.
- **Bloat:** long-running queries hold back vacuum (old row versions cannot be cleaned up), so tables bloat.
- **Wrong shape:** the schema is normalized for writes, so reports need many joins.
- **Missing history:** production often keeps only current state, not the history analysts need.

A **read replica** is a simple first step: it moves the load off the primary. But it is still a row store, and long queries on it can still cause replication lag or be cancelled.

5 Moving the data: ETL vs ELT

ETL means extract, transform, then load: data is cleaned in a separate engine before it reaches the warehouse. **ELT** means extract, load the raw data, then transform **inside the warehouse** with SQL, using tools like **dbt**. Because warehouse compute is cheap and elastic, ELT has become the common modern approach.

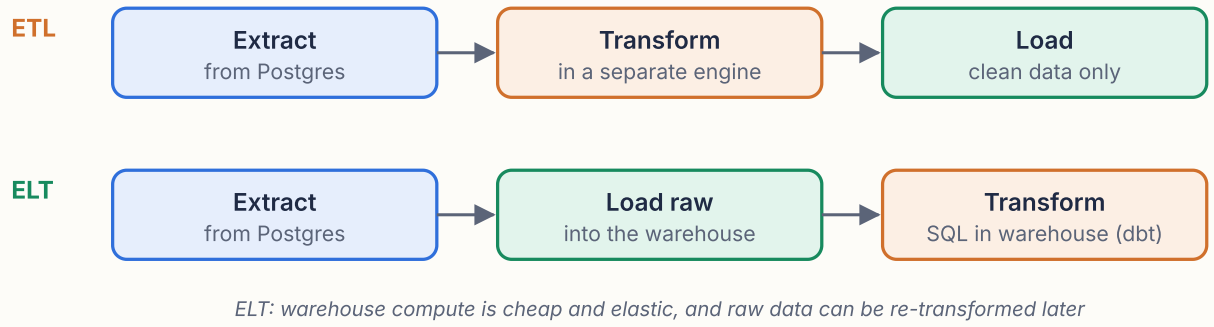


Figure 4. The same three steps in a different order. ELT keeps the raw data, so a fixed transformation can simply be re-run.

Back to the finance report. Order changes are copied into BigQuery, every few minutes with change data capture (next lesson) or nightly in batches. dbt models build a `revenue` table. The report and dashboards run there in seconds, and production checkout never notices.

```
-- runs in the warehouse, not on production
SELECT d.month, s.country, SUM(f.amount) AS revenue
  FROM fact_orders f
  JOIN dim_date  d ON d.date_key  = f.date_key
  JOIN dim_store s ON s.store_key = f.store_key
 WHERE d.year IN (2025, 2026)
 GROUP BY d.month, s.country;
```

COMMON MISTAKES

- Running heavy reports or BI dashboards against the production primary.
- Treating a read replica as a warehouse: it is still a row store.
- Using a column store for the app's point lookups and single-row updates.
- `SELECT *` in a columnar warehouse: you pay to read every column.

6 Key terms

OLTP: many small transactions on few rows

OLAP: few large scans and aggregations

Row store: each row stored together

Column store: each column stored together

Vectorized: process values in CPU batches

Data warehouse: columnar store for analytics

Star schema: fact table plus dimension tables

ETL: transform before loading

ELT: load raw, transform in warehouse (dbt)

7 Interview questions

Q1. Why not run analytics on the OLTP database? BASIC

Large scans compete with transactions for CPU, I/O and cache, slowing users. Long queries hold back vacuum and cause bloat. A normalized row store is inefficient for aggregating a few columns over billions of rows. Move the data to a columnar warehouse with ETL or ELT.

Q2. Why are column stores faster for analytics? BASIC

A query reads only the columns it uses, similar values compress 5–10×, and engines process them in vectorized batches. SUM over 1 of 20 columns reads about 1/20 of the data, less after compression.

Q3. What is the difference between ETL and ELT? BASIC

ETL transforms data before loading it. ELT loads raw data first and transforms it inside the warehouse with SQL (e.g. dbt). ELT is now common because warehouse compute is elastic and raw data stays available.

Q4. The finance report slows checkout every Monday at 9. Fix it, short and long term. SCENARIO

Short term: move it to a read replica, or off-peak. Long term: stream order changes with CDC (or nightly batch) into BigQuery or Snowflake, build a revenue table with dbt, and point the report there.

Q5. Design the schema for "revenue by month, country and product category". SCENARIO

A star schema: fact_orders (amount, quantity, date_key, store_key, product_key) and dimensions dim_date, dim_store (country) and dim_product (category). Join and GROUP BY month, country, category.

Q6. A dashboard needs data that is at most 5 minutes old. Nightly batch or CDC? SCENARIO

CDC: it streams committed changes continuously, so the warehouse lags by seconds to minutes. A nightly batch would be up to 24 hours stale.

Q7. Why not use a column store for the app itself? TRICKY

Fetching or updating one order touches every column file, and frequent single-row updates are slow, because columnar data is stored in large compressed blocks. Row stores win at point reads and writes.

Q8. Is a read replica enough for analytics? TRICKY

It isolates the primary's CPU, but it is still a row store, scans are still slow, long queries can be cancelled or cause lag, and it holds no history. Fine as a first step, not a warehouse.

Next: SD-E-031 · Change Data Capture (CDC) backendroadmap.com